1. 文件浏览器

主要学习 QDir类、QListWidget类、QProcess类 的使用

分析实现思路：

```
1.界面设计：
    一个按钮：返回上一级目录
    一个单行文本框：显示当前目录的完整路径
    一个 QListWidget：显示当前目录下所有文件夹以及文件列表
    注意：在此采用了布局的方式设计界面，更符合实际
        按钮和文本框放在一个水平布局中 QHBoxLayout
        水平布局和 QListWidget 放在一个垂直布局中 QVBoxLayout
2.初始化当前目录到当行文本编辑框中
3.得到当前的目录路径
4.读取这个目录下所有文件夹以及文件信息列表
5.通过循环的方式给 QListWidget 中挨个添加文件夹以及文件信息（文件夹或者文件的名称）
6.选择列表中的选项双击：
    如果是文件夹，切换目录，重复3.4.5步骤
    如果是文件，就使用记事本的程序打开这个文件
        注意： python中，在一个程序中启动另外一个py程序，只能用命令的方式。
        格式： python  test.py 文件路径

7.点击返回上一级目录按钮：
    如果能切换目录成功，重复3.4.5步骤
    否则：什么都不做
```

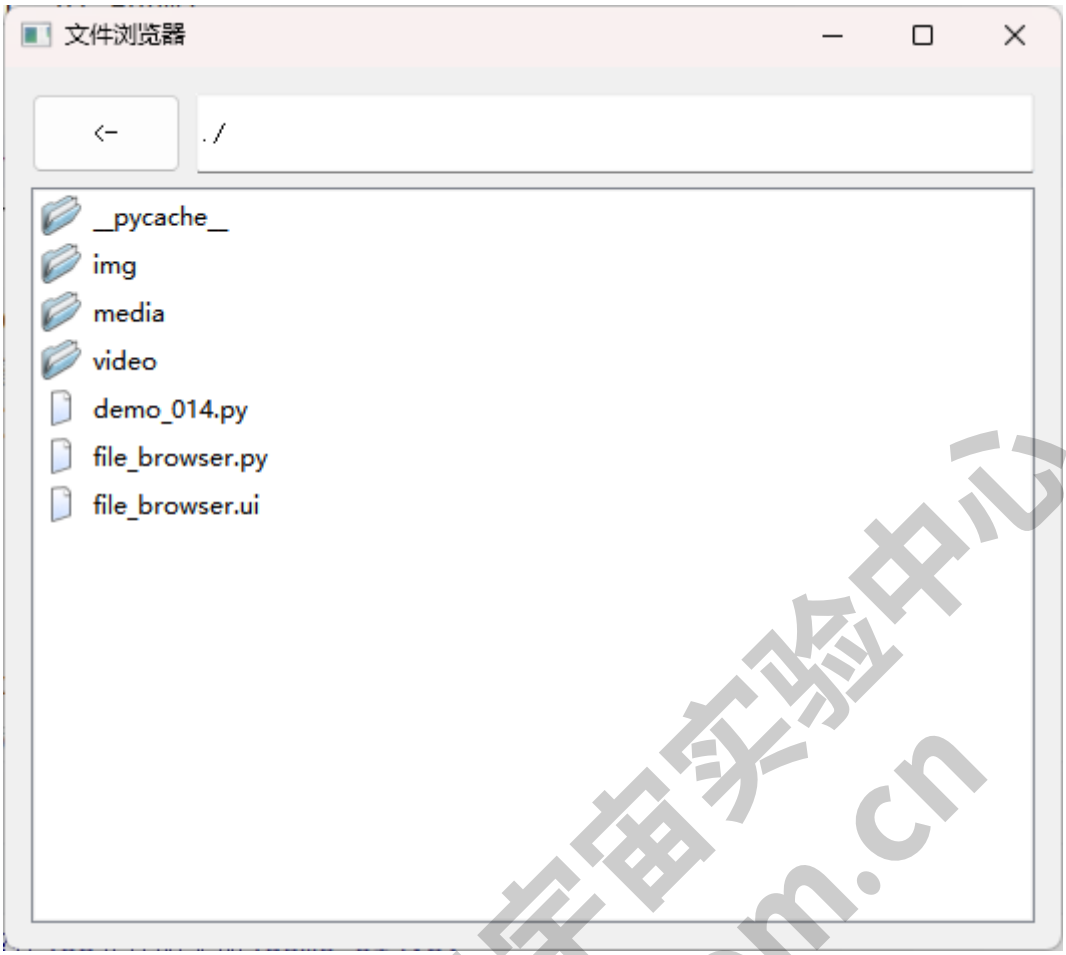
用到的类以及方法：

```
QDir
    setPath()        设置路径
    entryInfoList() 获取当前目录下所有文件夹以及文件信息，返回一个列表
    cd()       切换目录
    cdUp()   返回上一级目录
    absolutePath()   获取当前目录的绝对路径
QListWidget
    addItem()    添加单元项
    clear()      清空列表
    itemDoubleClicked()      单元项双击信号
QListWidgetItem
    setText()    设置文本
    setIcon()    设置图标
    text()       获取文本内容
QFileInfoList
    at()         访问元素
QFileInfo
    fileName()   获取文件名
    isDir()      判断是不是文件夹
    isFile()     判断是不是文件
QProcess
    start()      启动进程
```

代码实现：

华清远见|元宇宙实验中心
yyzlab.com.cn

使用ui的方式+布局来实现



1.1 预定义的筛选器

它们可以用来过滤目录列表中的文件或子目录。

```
QDir.AllEntries:        列出目录中的所有条目，包括文件、子目录和符号链接。
QDir.AllDirs:           仅列出目录中的目录（不包括文件）。
QDir.Files:             仅列出目录中的文件。
QDir.Drives:            列出系统中的所有根目录（例如，在 Windows 上的 C:\ 和 D:\）。
QDir.NoSymLinks:        不列出符号链接。
QDir.NoDot:             不列出 “.” 条目（当前目录）。
QDir.NoDotDot:          不列出 “…” 条目（父目录）。
QDir.NoDotAndDotDot:    不列出 “.” 和 “…” 条目。
QDir.Hidden:            列出隐藏文件和目录。
QDir.System:            列出系统文件和目录。
QDir.CaseSensitive:     文件名比较是区分大小写的
```

2. 总结

2.1 QDir

在 PyQt5 中，`QDir` 是一个用于操作文件系统目录的类。它属于 `PyQt5.QtCore` 模块，主要用于处理与目录相关的操作，例如列出目录内容、创建或删除目录、检查路径是否存在等。`QDir` 提供了一种跨平台的方式来访问和管理文件系统中的目录。

`QDir` 的主要作用是提供对文件系统目录的操作，通常用于以下场景：

- **列出目录内容**：获取指定目录下的文件和子目录列表。

华清远见元宇宙实验中心 yvlab.com.cn

- **创建和删除目录**：动态创建或删除目录。
- **检查路径状态**：判断路径是否存在、是否为文件或目录。
- **路径操作**：解析和操作路径（如绝对路径、相对路径等）。
- **过滤文件**：根据文件名模式（如通配符）筛选文件。

2.1.1 `QDir` 的导入方式

```
from PyQt5.QtCore import QDir
```

2.1.2 常用方法

2.1.2.1 创建 `QDir` 对象

- `QDir(path="")`：通过路径创建一个 `QDir` 对象。

  ```
  dir = QDir("/path/to/directory")
  ```

2.1.2.2 列出目录内容

- `entryList(filters=QDir.NoFilter, sort=QDir.NoSort)`：获取目录中的文件和子目录列表。
  - 参数：
    - `filters`：过滤器，用于筛选文件或目录（如 `QDir.Files` 表示仅列出文件）。
    - `sort`：排序方式（如 `QDir.Name` 表示按名称排序）。

  ```
  files_and_dirs = dir.entryList()
  print(files_and_dirs)
  ```

2.1.2.3 检查路径状态

- `exists()`：检查目录是否存在。

  ```
  if dir.exists():
      print("目录存在")
  ```

- `isReadable()`：检查目录是否可读。

  ```
  if dir.isReadable():
      print("目录可读")
  ```

- `isAbsolute()`：检查路径是否为绝对路径。

  ```
  if dir.isAbsolute():
      print("这是绝对路径")
  ```

2.1.2.4 创建和删除目录

- `mkdir(str dirName)`：在当前目录下创建子目录。

  ```
  dir.mkdir("new_directory")
  ```

华清远见|元宇宙实验中心 yyzlab.com.cn

- `rmdir(str dirName)`：删除指定的子目录（目录必须为空）。

```
dir.rmdir("new_directory")
```

2.1.2.5 路径操作

- `absolutePath()`：获取目录的绝对路径。

```
abs_path = dir.absolutePath()
print(f"绝对路径: {abs_path}")
```

- `relativeFilePath(str filePath)`：将文件路径转换为相对于当前目录的路径。

```
rel_path = dir.relativeFilePath("/path/to/file.txt")
print(f"相对路径: {rel_path}")
```

- `cd(str dirName)`：切换到指定的子目录。

```
dir.cd("subdirectory")
```

- `cdUp()`：切换到上一级目录。

```
dir.cdUp()
```

2.1.2.6 过滤文件

- `setNameFilters(list filters)`：设置文件名过滤器。

```
dir.setNameFilters(["*.txt", "*.py"])  # 仅列出 .txt 和 .py 文件
filtered_files = dir.entryList()
print(filtered_files)
```

2.1.2.7 获取根目录

- `root()`：获取系统的根目录。

```
root_dir = QDir.root()
print(f"根目录: {root_dir.path()}")
```

2.1.3 典型应用场景

- **文件浏览器**：列出目录内容并显示给用户。
- **动态目录管理**：创建或删除目录以满足程序需求。
- **路径解析**：解析和操作路径（如绝对路径、相对路径）。
- **文件过滤**：根据文件类型或其他条件筛选文件。
- **跨平台开发**：确保代码在不同操作系统上的路径操作一致性。

2.2 QProcess

华清远见元宇宙实验中心 yyzlab.com.cn

在 PyQt5 中，`QProcess` 是一个用于启动和与外部进程交互的类。它属于 `PyQt5.QtCore` 模块，主要用于执行外部程序（如命令行工具、脚本等）并与之通信。`QProcess` 提供了跨平台的方式启动子进程，并支持读取子进程的标准输出和标准错误流，以及向子进程写入输入数据。

`QProcess` 的主要作用是启动外部程序并与其进行交互，通常用于以下场景：

- **运行外部命令**：执行系统命令或脚本。
- **捕获输出**：获取外部程序的标准输出和标准错误流。
- **发送输入**：向外部程序的标准输入流写入数据。
- **监控进程状态**：检查进程是否正在运行、已退出等。
- **跨平台支持**：确保代码在不同操作系统上的兼容性。

2.2.1 `QProcess` 的导入方式

```
from PyQt5.QtCore import QProcess
```

2.2.2 常用方法

2.2.2.1 启动进程

- `start(program, arguments=[])`：启动指定的外部程序。
  - 参数：
    - `program`：要运行的程序路径或命令。
    - `arguments`：传递给程序的参数列表。

```
process = QProcess()
process.start("ping", ["www.google.com"])
```

- `execute(program, arguments=[])`：同步执行外部程序并等待其完成。

```
exit_code = QProcess.execute("ls", ["-l"])
print(f"退出码: {exit_code}")
```

2.2.2.2 读取输出

- `readAllStandardOutput()`：读取子进程的标准输出。

```
output = process.readAllStandardOutput().data().decode()
print(f"标准输出: {output}")
```

- `readAllStandardError()`：读取子进程的标准错误输出。

```
error = process.readAllStandardError().data().decode()
print(f"标准错误: {error}")
```

2.2.2.3 写入输入

- `write(data)`：向子进程的标准输入流写入数据。

```
process.write(b"input data\n")
```

华清远见元宇宙实验中心 yyzlab.com.cn

- `closeWriteChannel()`：关闭写入通道，通知子进程输入结束。

```
process.closeWriteChannel()
```

2.2.2.4 监控进程状态

- `waitForStarted(msecs=30000)`：等待进程启动。

```
if process.waitForStarted():
    print("进程已启动")
```

- `waitForFinished(msecs=30000)`：等待进程结束。

```
if process.waitForFinished():
    print("进程已结束")
```

- `state()`：获取进程的当前状态。
  - 返回值：
    - `QProcess.NotRunning`：进程未运行。
    - `QProcess.Starting`：进程正在启动。
    - `QProcess.Running`：进程正在运行。

```
if process.state() == QProcess.Running:
    print("进程正在运行")
```

- `exitCode()`：获取进程的退出码。

```
print(f"退出码: {process.exitCode()}")
```

2.2.2.5 信号与槽

- `started.connect(slot)`：当进程启动时触发。

```
process.started.connect(lambda: print("进程已启动"))
```

- `finished.connect(slot)`：当进程结束时触发。

```
process.finished.connect(lambda exit_code: print(f"进程已结束，退出码:
{exit_code}"))
```

- `readyReadStandardOutput.connect(slot)`：当有标准输出可读时触发。

```
process.readyReadStandardOutput.connect(lambda:
print(process.readAllStandardOutput().data().decode()))
```

- `readyReadStandardError.connect(slot)`：当有标准错误输出可读时触发。

```
process.readyReadStandardError.connect(lambda:
print(process.readAllStandardError().data().decode()))
```

华清远见元宇宙实验中心
yyzlab.com.cn

2.2.3 典型应用场景

- **运行系统命令**：执行命令行工具（如 `ping`、`ls` 等）。
- **与脚本交互**：运行 Python 脚本或其他语言的脚本，并捕获其输出。
- **实时监控输出**：实时读取子进程的标准输出和标准错误流。
- **自动化任务**：通过外部程序实现自动化任务（如文件转换、数据处理等）。
- **跨平台开发**：确保代码在不同操作系统上的兼容性。

2.3 QListWidgetItem

在 PyQt5 中，`QListWidgetItem` 是一个用于表示 `QListWidget` 中的列表项的类。它属于 `PyQt5.QtWidgets` 模块，主要用于定义和管理列表控件中的每一项内容。`QListWidgetItem` 提供了丰富的功能，例如设置文本、图标、复选框状态等，并支持与用户交互。

`QListWidgetItem` 的主要作用是为 `QListWidget` 提供单个列表项的功能，通常用于以下场景：

- **显示文本**：在列表中显示简单的文本内容。
- **显示图标**：为列表项添加图标，增强视觉效果。
- **支持复选框**：为列表项添加复选框，实现多选功能。
- **动态更新**：可以在运行时动态修改列表项的内容或状态。
- **数据绑定**：可以为每个列表项附加自定义数据。

2.3.1 `QListWidgetItem` 的导入方式

```
from PyQt5.QtWidgets import QListWidgetItem
```

2.3.2 常用方法

2.3.2.1 创建列表项

- `QListWidgetItem(str text, parent=None)`：通过文本创建一个列表项。

  ```
  item = QListWidgetItem("示例项")
  ```

- `QListWidgetItem(QIcon icon, str text, parent=None)`：通过图标和文本创建一个列表项。

  ```
  from PyQt5.QtGui import QIcon

  item = QListWidgetItem(QIcon("icon.png"), "带图标的项")
  ```

2.3.2.2 设置文本

- `setText(str text)`：设置列表项的文本。

  ```
  item.setText("新的文本")
  ```

- `text()`：获取列表项的文本。

  ```
  print(item.text())
  ```

华清远见元宇宙实验中心 yyzlab.com.cn

2.3.2.3 设置图标

- `setIcon(QIcon icon)`：为列表项设置图标。

```
item.setIcon(QIcon("new_icon.png"))
```

- `icon()`：获取列表项的图标。

```
icon = item.icon()
```

2.3.2.4 设置复选框状态

- `setCheckState(Qt.CheckState state)`：设置列表项的复选框状态。
  - 参数：
    - `Qt.Unchecked`：未选中。
    - `Qt.PartiallyChecked`：部分选中（仅适用于三态复选框）。
    - `Qt.Checked`：已选中。

```
from PyQt5.QtCore import Qt

item.setCheckState(Qt.Checked)
```

- `checkState()`：获取列表项的复选框状态。

```
state = item.checkState()
if state == Qt.Checked:
    print("列表项已选中")
```

2.3.2.5 附加自定义数据

- `setData(int role, QVariant value)`：为列表项附加自定义数据。
  - 参数：
    - `role`：数据的角色（如 `Qt.UserRole`）。
    - `value`：要附加的数据。

```
item.setData(Qt.UserRole, "自定义数据")
```

- `data(int role)`：获取列表项的自定义数据。

```
custom_data = item.data(Qt.UserRole)
print(custom_data)
```

2.3.2.6 设置工具提示

- `setToolTip(str tip)`：设置列表项的工具提示信息（鼠标悬停时显示）。

```
item.setToolTip("这是一个示例项")
```

华清远见元宇宙实验中心 yyzlab.com.cn

2.3.2.7 启用或禁用项

- `setFlags(Qt.ItemFlags flags)`：设置列表项的标志位（如是否可选、是否可编辑）。

```
from PyQt5.QtCore import Qt

item.setFlags(Qt.ItemIsSelectable | Qt.ItemIsEnabled)  # 设置为可选且可用
```

2.3.3 典型应用场景

- **显示列表内容**：在 `QListWidget` 中显示文本或带图标的列表项。
- **多选功能**：通过复选框实现多选功能。
- **动态更新**：在运行时动态修改列表项的内容或状态。
- **数据绑定**：为每个列表项附加自定义数据，方便后续处理。
- **增强交互性**：通过工具提示、图标等提升用户体验。

华清远见元宇宙实验中心
yyzlab.com.cn